

Omphalos: the report to read before the meeting

ACE, proof search, spending, and what we learned

13 September 2026 · Evidence through 12 September

1. What matters most

The strongest overall result is progress on **controlling the cost of proof search**. We have also implemented and investigated ACE extensively, but a large, general improvement in theorem coverage is still unestablished. Those conclusions fit the thesis: Delphyné’s control and budgeting mechanisms are part of the research question.

ACE means **Agentic Context Engineering**. Our prover learns reusable advice from its attempts and places it in a persistent book that later attempts read. The model’s weights do not change. The important question is whether this extra context helps solve more problems, solve them more cheaply, or both.

There are three results to remember:

- **A useful budget trade-off:** the first bounded configuration solved 25/40 validation problems for \$0.6780, against the older X3 comparator’s 28/40 for \$1.5238. Spending fell 55.5%, with three fewer solves. You accepted that trade-off. Its original no-coverage-loss gate still failed.
- **A promising matched ACE comparison:** within the current bounded Luna program, its own book gives 27/40 validation solves against 24/40 without it, for \$0.6672 against \$0.7765. Three gains and no losses is encouraging, but the paired result is inconclusive: $p=0.25$.
- **A limit on the interpretation:** the book’s lower recorded bill depends on cached-input pricing. Charging the same recorded inputs entirely as uncached reverses Luna’s cost difference into a small increase.

Several experiments failed for reasons more specific than “ACE is bad” or “the model is weak.” Advice was sometimes based on incomplete execution evidence. A reducer introduced invalid executable syntax. A response parser prevented valid outputs from becoming candidates. Some apparently successful treatments were never exposed in the relevant requests. Other paths simply repeated incomplete mathematical arguments.

My current priority for the next work: diagnose lost opportunity in the bounded controller and improve proof completion; then repair the evidence supplied to learning. More powerful writers and more elaborate advice should earn their place through a demonstrated mechanism.

This document is your reading version. It groups the experiments and keeps the explanations needed to understand them. The advisor deck is shorter and uses bullets. The original 41-page report, detailed tables, and workbook remain available for reference. No new experiments were run for these materials. testX and protected challenge evidence are excluded; validationX is repeatedly reused development data.

2. ACE: the complete loop in plain language

A Rocq proof attempt produces more than a success or failure. It can reveal which lemma exists, which syntax the environment accepts, which representation makes arithmetic automation work, or which partial argument still leaves an obligation. ACE tries to turn that experience into reusable context.

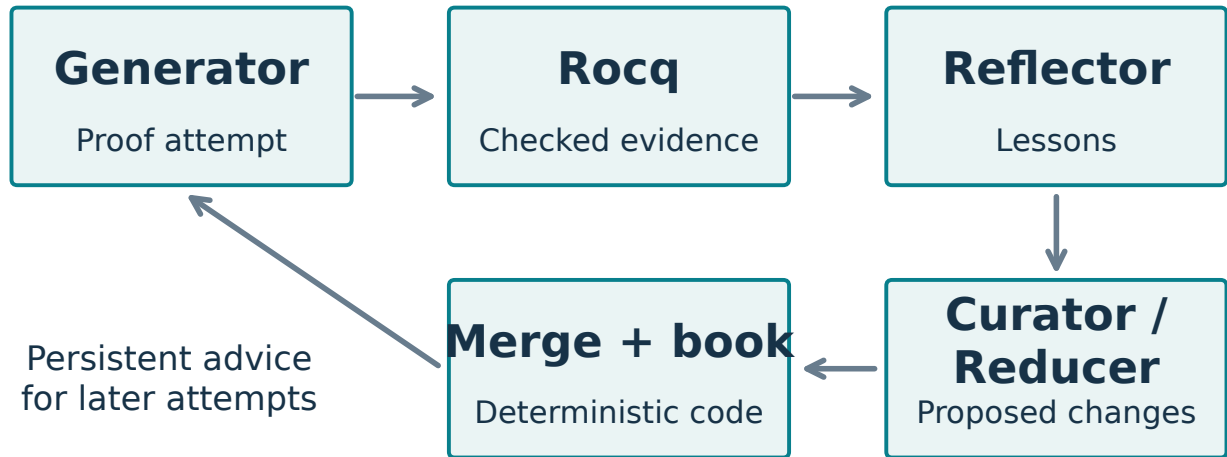


Figure 1: The learning loop. LLM roles propose lessons; deterministic code owns the persistent book.

The **Generator** proposes a proof using the current book and available tools. Rocq checks the proposal. The **Reflector** interprets the attempt: what worked, what failed, and what might transfer. The **Curator** proposes entries or changes. In our batched implementation, a **Reducer** chooses among several proposals. Ordinary Python code then merges the changes, assigns identifiers, and saves the book.

Deterministic updates preserve individual bullets and make changes auditable. They do not certify that the advice is true or useful.

A valid lemma name can still occur in an invalid snippet. Correct advice can also be irrelevant or arrive too late.

There are several learning settings:

Setting	What changes during evaluation?	Main interpretation
Offline	The learned book is frozen	Does this particular artifact transfer?
Cold online	The book starts empty and evolves	Does experience help later tasks in this order?
Warm online	A learned book keeps evolving	Does prior advice help this new sequence?
Multiple epochs	Training examples are revisited	Does further curation help on repeated evidence?

Online order matters; repeated epochs do not create independent theorems. Success ultimately means **a completed proof accepted by Rocq**. Better explanations and correct local rewrites are intermediate observations, which need not increase coverage.

3. How we implemented it in Delphyne

Delphyne separates a **strategy**, which describes possible application steps, from a **policy**, which chooses how to explore them and spend resources. Our code expresses model choices through queries and verification through Compute operations. This gives us distinct places to change advice, search behavior, and resource control.

The original ACE proposal query extends the agentic proof query with the rendered book. That text is carried in the query itself, making the request self-contained for caching. ACE has its own templates and query identity; the example selector connects it to the existing agentic demonstrations. Newer bounded/focused queries have different contracts, so they must not be treated as the same baseline with one extra paragraph.

The book contains structured bullets with identifiers, sections, and version-dependent usage information. Reflection and curation produce structured proposals. Deterministic code applies updates, refinement, and deduplication. Later variants use reference checks and evidence-oriented prompts to discourage invented names and unsupported generalizations. The Reducer makes batched adaptation practical by choosing a smaller set of additions.

Storage is content-addressed: canonical YAML is saved under its hash, and experiment configurations record the exact book they consumed. This avoids confusing a mutable step number with an immutable artifact. Render versions, model options, seeds, and upstream evidence also matter. A cached answer does not prove that a historical training chain can be reconstructed under today’s code.

The newer prover works with typed outcomes, bounded goal views, checked prefixes, and explicit proof operations. Its controls include money admission, request limits, verifier-time allowances, and optional recovery or stopping behavior. A campaign ledger reserves paid requests before dispatch and settles recorded charges afterward. Unknown charges remain liabilities instead of disappearing from the bill.

Intervention	Where it belongs	What it can change
Learned advice	Query context and book	What the model knows at a decision
Search or recovery	Strategy/policy composition	Which attempts get explored
Verifier operation	Budgeted Compute and bridge	What is checked and what it costs
Admission rule	Budget stream and transport adapter	Whether the next request may run
Example selection	Demonstration selection	Which worked examples reach a query

This is why attribution is difficult. A candidate that changes all these pieces is a new program. Its lower bill does not identify which piece caused the saving. A clean ACE ablation holds the rest of the program fixed and removes the book and its conditional instructions.

Private warm Rocq servers, stable paths, bounded replies, memoization, and supervised workers are substantial engineering achievements. They make experimentation feasible. Their historical microbenchmarks describe local operations, not equivalent reductions in model bills or whole-benchmark runtime.

4. How to read the measurements

Always read **coverage, total cost, and cost per qualified solve together**. The cost per solve divides the bill for the entire panel by its qualified successes. Unsuccessful attempts belong in the numerator. Removing a failed or expensive cell makes the comparison easier to win for the wrong reason.

The X development panels contain forty theorems each. Two seeds give eighty observations, not eighty independent theorems. `trainX` supports adaptation and tuning. `validationX` has been consulted repeatedly and is now development data. `testX` remains closed, as does protected challenge evidence.

The early canonical Luna program used a nominal \$0.05 allowance. X experiments use \$0.10. The recent Terra comparisons use \$1.00 under the recorded tenfold tariff ratio. These are different experimental settings. Older stopping policies could cross a nominal allowance; conservative admission can instead stop with money still unspent.

Term	What it means here
Raw success	The run recorded a completed proof
Qualified success	It also satisfies that study's cost rule
Recorded spending	Charges for observed requests and attempts
Token repricing	Recorded usage valued using an explicit dated tariff
Price sensitivity	The same usage valued under a hypothetical tariff
Preparation cost	Generating training experience and constructing a book
Continuation cost	Original paid prefix plus the new continuation, once

Campaign receipts and repriced archive summaries are not interchangeable. Nor should a carried reference be counted as a new research expense. The audited subtotal across thirteen accessible receipt exports is **\$88.7639**. It excludes older or closed campaigns, incomplete preparation charges, assistant usage, and engineering time. It is not a lifetime research bill.

For statistical support, current project conventions use two-sided $p < 0.10$ and 90% intervals, with repeated measurements and related theorem families handled appropriately. A pilot can still be worth extending when it misses that threshold. Conversely, a favorable aggregate does not establish a robust improvement.

Keep four decisions separate: **what was observed; whether the evidence supports a statistical claim; whether the trade-off is useful; whether the default should change**. The bounded configuration is the clearest example: a useful accepted trade-off can coexist with a failed original gate.

Historical controls also differ in provider time, sampling, caching, and sometimes controller behavior. We can use them honestly, but cannot silently label all historical differences as treatment effects. Repeating identical configurations has produced disagreements, so small gains need both uncertainty and an exposure check: did the intervention actually reach the request or operation where the gain occurred?

5. The early ACE experiments: substantial machinery, small effects

The initial replication established that the loop worked. ACE changed the prover’s behavior more clearly than its final coverage. On the original validation panel, both programs solved 32/40 observations. The non-ACE bill was \$0.4446 and the ACE bill \$0.5058. Output tokens fell roughly by half, but the larger context offset that saving. The two arms exchanged one gain and one loss.

The paper’s dramatic context-collapse behavior did not reproduce in our setting. A separate input defect did matter: definition preambles were omitted from some problems. Restoring them gained one solve in an eight-cell diagnostic. That fix is part of the environment contract, not an ACE win, and should not be repeatedly re-ablated.

The later X studies introduced larger development panels and more complete adaptation machinery. The following table groups their main lessons; historical totals should be read under each study’s original scoring.

Family	What we changed	Main observation
v2 offline and online	Structured updates, evolving books, warm starts	Frozen offline tied at 54/80; warm starts did not reliably help
X3	Reference-oriented curation and batched reduction	A leading rendering reached 57/80; no-reflection also reached 57/80
More epochs / stronger writers	Repeated adaptation or more expensive writing roles	More work did not consistently improve coverage
v5	Failure digests, grounding, and curation audits	Better checks and diagnoses, without a clear coverage gain
Rendering and advice on error	Different presentation and selective delivery	A rendering advantage reversed across books; initial generalization was retracted
Repair-mined advice	Advice derived from training repairs	Some useful content, without an established general gain

The no-reflection result is important but narrow: it limits the demonstrated value of that particular reflection stage in that experiment. It does not prove that all reflection is useless. The same applies to stronger writers: their failure to win does not show that the writer model can never matter.

The deterministic digest-table comparison suggested that curation sometimes selects useful content better than mechanically listing failures. But better curation still has to produce advice that is relevant, valid, exposed, and actionable on later tasks.

Several historical chains have provenance limitations: stale-directory recovery, changed upstream books, or early taxonomy versions. The full report preserves those details. For the meeting, the defensible story is that we implemented and tested many ACE mechanisms, found modest and inconsistent coverage effects, and learned that book quality alone is an incomplete explanation of end-to-end performance.

6. Budgeting: the clearest accepted trade-off

The first bounded campaign used explicit money control and focused proof operations. It cost **\$0.6780** for **25/40 qualified validation solves**, compared with **\$1.5238** for **28/40** from its older X3 reference. That is 55.5% lower total spending. Whole-panel cost per solve fell from about \$0.0544 to \$0.0271.

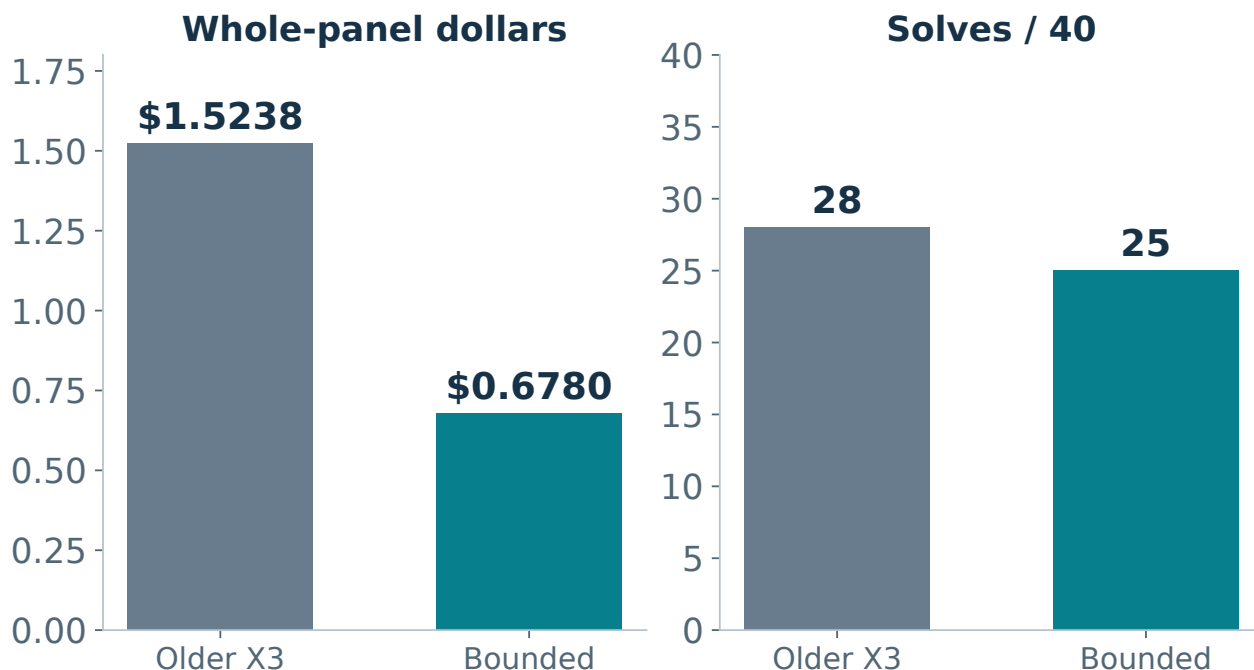


Figure 2: The original budget comparison. Both bars include spending on unsuccessful attempts; the lower bill comes with three fewer solves.

This is a useful result for a thesis about resource control. It is not evidence of preserved coverage. The original gate required no coverage loss and failed; your subsequent acceptance concerns the value of the trade-off. Both facts belong in the account.

A control-cycle follow-up recovered 28/40 solves for \$1.0253. Relative to the bounded reference, it spent about 51.2% more for three additional solves. It did not establish a general improvement over all comparators. The surviving implementation is also incomplete, which limits a current-source reproduction claim.

The polish campaign added richer interfaces and controls. Across eighty validation observations, both arms solved 53, while polish cost \$1.5129 against \$1.4333: about 5.55% more. Its changes remain useful implementation options or diagnostic aids, but the benchmark did not justify a coverage or cost win.

The old stall work and later bounded work address different resource problems. A stall policy tries to stop unproductive repetition. Admission decides whether a request may begin. A verifier limit bounds checking work. A request limit bounds interactions. They can complement one another, but bundling them makes the cause of a gain or loss harder to identify.

The next question is where the bounded program loses useful opportunity. The newer no-book baseline already loses coverage against the earlier no-book program. That loss cannot be caused by ACE. Before adding more advice, we should examine the controller, feedback, and proof-submission behavior that changed with it.

7. Local repairs and model effort: promise did not consistently transfer

Some studies measured local transitions rather than completed theorems. Those studies are useful for diagnosing a mechanism, but their success counts cannot be copied into a benchmark table as solved problems.

Study	What it measured	What happened
Applicability	Whether advice helps at selected proof states	Forty-eight local episodes; fixed examples produced three useful transitions versus two for the tested alternative
Checked change	Whether a proposed representation change is correct and useful	Twenty-four episodes; checked arm 6/6 correct versus 3/6, and about 43% cheaper
Effort probes	How reasoning effort affects a small diagnostic panel	Higher effort helped some Luna cases; more effort was not monotonically better
Model/role suite	Which model and effort should perform each role	Promising local choices did not reliably improve full-panel coverage

The applicability guard itself could reject a valid separate-sentence suffix before Rocq checked it. “The guard refused the candidate” and “Rocq rejected the proof” are different failure explanations. In the checked-change study, a correct conversion could still leave the main mathematical task unfinished.

The model suite initially produced an encouraging selection pilot: 18/24 successes against 17/24 at lower cost. It missed a predeclared two-solve gain. Treating that missed pilot gate as a blanket veto was later corrected. A small promising effect can justify a bounded follow-up without being promoted to a confirmed result.

The authorized full contenders gave a less favorable picture:

Configuration	Train solves / 40	Validation solves / 40	Interpretation
Current reference	28	27	Comparison anchor
Medium Reflector contender	26	19	Substantial validation loss
xhigh Reflector contender	27	24	Cheaper, with lower coverage

The xhigh Reflector contender saved about 15.6% on training and 12.2% on validation, but lost one and three solves respectively. It is another potential cost/coverage trade-off, not an unqualified quality improvement.

The practical lesson is to distinguish **model capacity**, **role quality**, **local correctness**, and **final proof coverage**. A writer can produce more polished advice while giving the Generator little actionable help. A solver can reason longer without crossing the missing formal step. A local repair can be valid without restoring a complete proof. Each stage needs its own measurement, followed by an end-to-end comparison when the mechanism merits it.

8. Mechanisms and coverage: why plausible fixes failed

The mechanisms campaign tested reasoning changes and structured recovery interfaces. One arm improved training slightly, reaching 29/40 against 28/40, but its validation total was 43/80 against 53/80. Its modest cost saving did not compensate for that coverage loss on cost per solve.

Three structured arms were confounded by response parsing. When a response contained several final messages, concatenation could turn individually valid JSON objects into invalid “extra data.” Fifty-one parsing failures were identified; thirty-three contained duplicated identical objects. A corrected last-final-message adapter recovered one already-paid training continuation offline. That is evidence of a real interface defect, not a reason to award every failed cell a hypothetical solve.

Fixed-book coverage treatments next tried different example selection and delivery. Their accessible validation results were 26/40 and 25/40 against the reference’s 27/40. One was somewhat cheaper; neither established a general coverage improvement. The protected stage is outside these materials.

Four isolated development follow-ups then tested improved demonstrations, bounded exploration, checked syntax repair, and history compaction:

Treatment	Train solves / 40	Validation solves / 40
Reference	28	27
Updated demonstration selection, D2	27	25
Bounded exploration, E2	28	24
Checked syntax repair, S	29	25
History compaction, H	27	25

All four validation bills were higher than the reference. A training saving or a locally valid repair was therefore insufficient evidence of useful transfer.

Exposure is crucial. Some gained or lost cases never received the extra example, feedback, or repair that supposedly explained the difference. Identical first requests sometimes produced different answers. Such disagreements belong in the observed treatment result, but they weaken a causal explanation based on a feature that never ran.

History compaction revealed a particularly useful failure. Shorter contexts led some paths to rediscover tool facts repeatedly. Five unsolved training cells never submitted a proof despite dozens of requests. A smaller prompt can make one request cheaper while making the entire search worse. Durable findings, failed attempts, checked prefixes, and a reliable return to submission matter more than transcript length alone.

For future work, the relevant order is: make the interface correct, show that the intended operation is reached, show that it makes useful progress, then measure whole-panel coverage and spending. The operational losses remain real even when an experiment gives weak evidence about the repaired mechanism’s potential.

9. The current ACE comparison—and the baseline trap

There is more than one non-ACE baseline. The early X program, the current bounded/focused program, and the current program with a book differ in important ways.

Seed-0 program	Train / 40	Validation / 40	Validation cost
Earlier agentic X, no book	33	27	\$0.8870
Current bounded/focused, no book	28	24	\$0.7765
Current bounded/focused, Luna book	28	27	\$0.6672

The new no-book program loses five training and three validation solves against the old no-book program. ACE cannot explain those losses. Within the current program, adding the Luna book restores three validation solves. Against the earlier program, the current flagship matches validation coverage at 24.77% lower recorded cost, while losing training coverage. Those are three distinct comparisons.

Qualified validation solves / 40

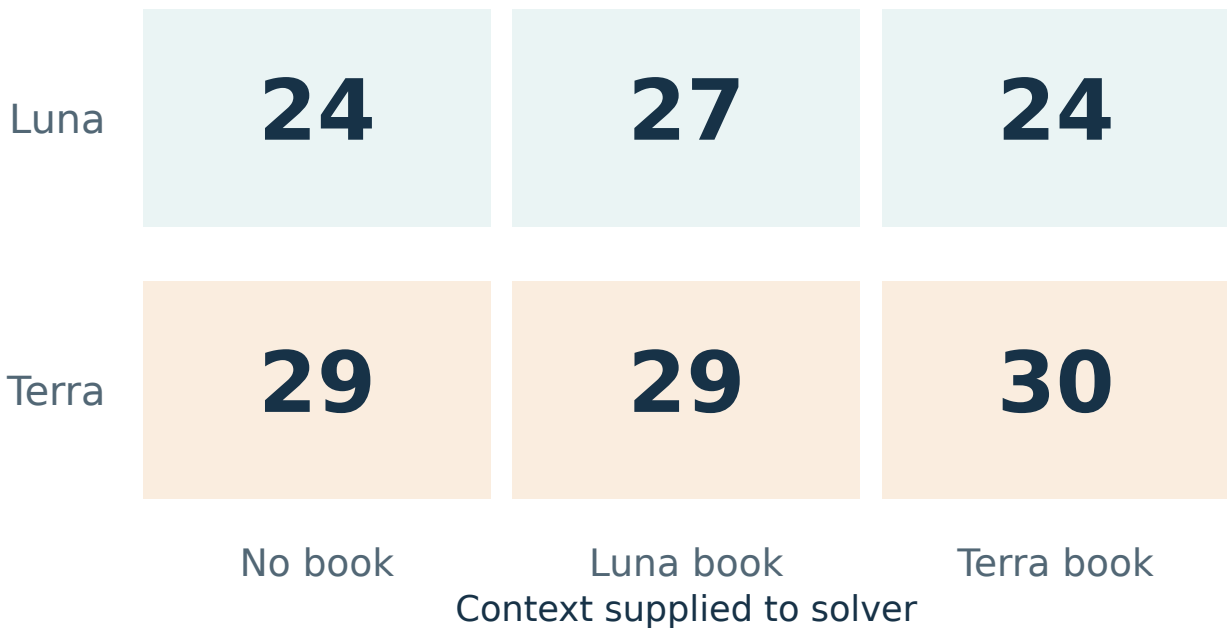


Figure 3: Full validation crossover: forty problems per solver/book combination. Terra solves more in absolute terms, but the same Luna book does not provide a larger increment on Terra.

The own-book validation comparison is 24→27 for Luna and 29→30 for Terra. Luna’s three gains and no losses give $p=0.25$. Terra’s two gains and one loss give $p=1.00$. The effects are modest and uncertain.

Holding the Luna book fixed gives a sharper interaction question. Luna gains three solves; Terra gains none. The difference between increments is -7.5 percentage points, with an inconclusive exact paired result, $p=0.375$. The study does not support the claim that a stronger solver generally unlocks more value from the same ACE book.

The crossover contains 240 observations over forty theorems, including reused records. These are not independent problems or all-new API episodes.

10. Spending versus problems solved

This is the most direct accounting view of the recent validation crossover. Each line processes the same forty problems in the fixed partition order. Moving right means more recorded spending; moving up means another qualified proof.

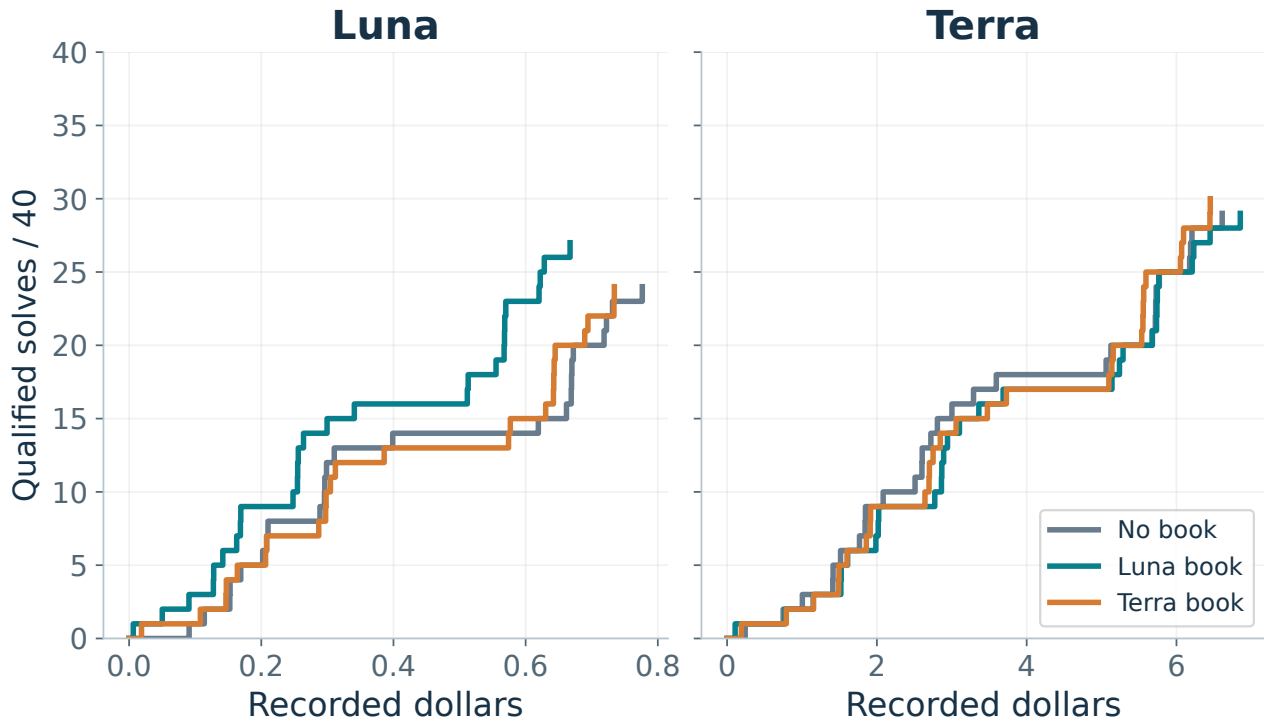


Figure 4: Observed cumulative spending. Failed attempts advance spending without increasing solves. The two panels deliberately use different dollar scales.

Luna without a book ends at 24 solves for \$0.7765. Its own book ends at 27 for \$0.6672; the Terra book gives 24 for \$0.7342. Terra without a book ends at 29 for \$6.6106; its own book gives 30 for \$6.4513, and the Luna book gives 29 for \$6.8523.

These curves summarize **observed trajectories**. They are not a scheduling policy. Concurrent invoices did not necessarily arrive in this order. Reordering tasks would change the intermediate shape; sorting successful cases by their eventual cost would require knowing outcomes in advance.

The endpoint is robust to that ordering and is the easiest comparison to remember. At recorded tariffs, Terra with its own book costs about 9.67 times as much as Luna with its own book for three additional validation solves. That may be a useful capacity comparison, but it does not establish an economical replacement for the Luna-class program.

The earlier report also contains cost-threshold curves. Those count observed successes whose final bills fall below a chosen threshold. They do **not** show what would happen if we reran every proof with that smaller budget. A changed allowance can alter admission, response length, tool use, and the proof path itself.

This distinction matters for the next experiment. We can ask how many existing successes qualify under a rule using recorded data. To learn whether a new budgeting policy spends less or finishes more proofs, we must evaluate that policy and record which requests it actually admits. A price sensitivity or a retrospective filter cannot supply those missing counterfactual executions.

11. Cache discounts and the cost of learning the book

The Luna own-book comparison lowers the observed validation bill by about **14.1%**. However, its cached-input share rises from about **64.6% to 75.9%**. That changes the economic interpretation.

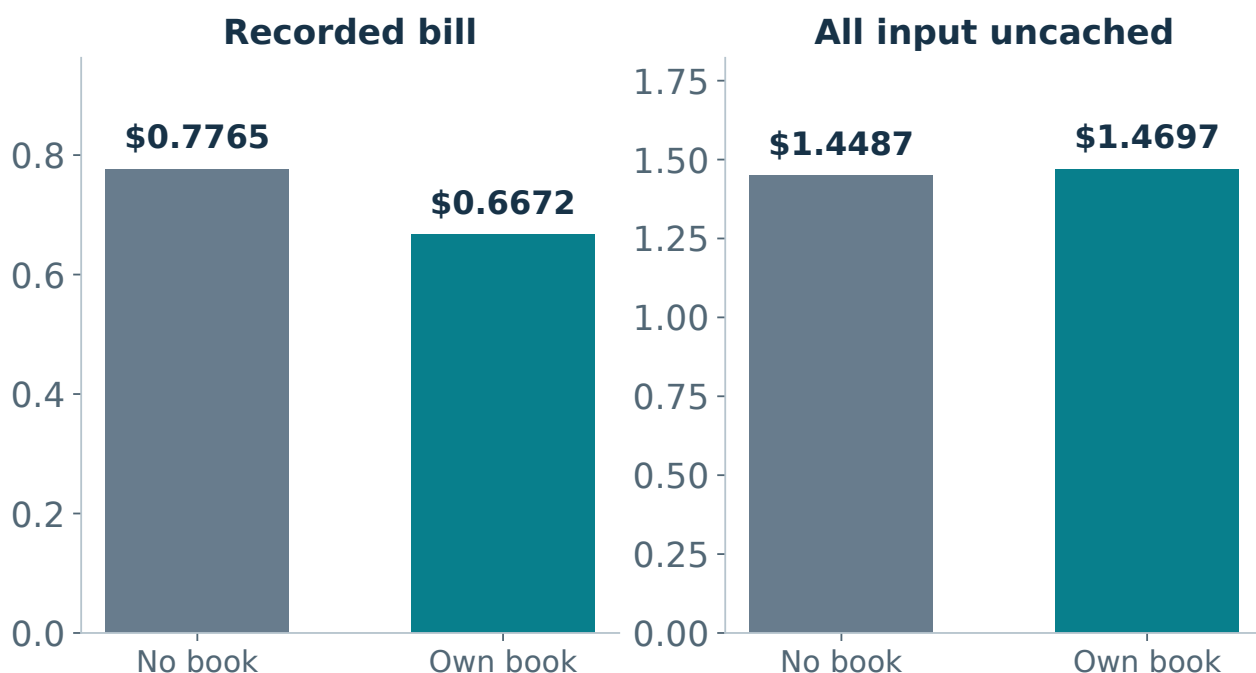


Figure 5: The same recorded Luna trajectories under two accounting rules. Removing the cache discount reverses the sign of the cost difference.

With all recorded input charged as uncached, Luna’s no-book bill is \$1.4487 and its own-book bill is \$1.4697: about **1.45% higher** with the book. Terra’s analogous increase is about **8.51%**. The measured saving is real at the recorded tariff, but it cannot all be described as less reasoning or less token usage.

This does not make caching a fake benefit. Reusing a persistent context can be economically valuable. It means the benefit depends on serving conditions and must be described at that level. Repricing Terra tokens at Luna rates is similarly a useful sensitivity calculation, not an available cheaper Terra service.

Book preparation is a separate bill. The known Luna preparation cost is about **\$0.9169**, with a missing historical embedding charge. Terra preparation is about **\$8.0992**, including its recorded embedding charge. Both include more than just the final frozen book: generating training experience and the writing roles belong in preparation.

The books contain twenty-six bullets, with estimated final sizes of 2,103 Luna tokens and 2,426 Terra tokens. During adaptation, Generator success was 33/40 and 38/40 respectively. Those evolving-book trajectories used a different controller from final bounded inference, so they cannot replace the frozen evaluation’s training counts.

Using the observed validation mean inference bills, the known preparation expense would break even after roughly **336 future Luna tasks** or **2,034 future Terra tasks**. Those are arithmetic scenarios assuming unchanged workload, coverage, prices, caching, and book usefulness. Luna’s figure is a lower bound because a historical charge is missing. Neither is a forecast of generalization.

12. Capacity, continuations, and unused money

The latest study combined a full solver/book crossover with matched writing jobs, small controlled diagnostics, and exact continuations of already-paid proof paths. These answer different questions. A stronger writer may improve a lesson; a stronger solver may finish a hard theorem; extra money may admit another useful attempt.

For selected eight-problem training panels, C denotes no ACE and E denotes the corrected own-book context. The continuation results were:

Solver / context	Initial solves	More money	Then more request/verifier allowance
Luna / C	4/8	4/8	4/8
Luna / E	3/8	3/8	3/8
Terra / C	4/8	6/8	6/8
Terra / E	3/8	7/8	7/8

The extra money recovered several Terra proofs. It did not recover more Luna proofs on these paths. Increasing request and verifier allowances afterward added none. But all eleven ordinary residual failures at the final level still stopped on money admission; a twelfth carried path was an API failure. An uncensored intrinsic model ceiling was not measured.

One Luna request was denied with about \$0.085742 left because its estimated reservation was about \$0.086368. A Terra example had \$0.798745 left against an estimate of \$0.813446. Money can remain unspent because the controller reserves a possible full response and a conservative input bound. That is a control-policy observation, not proof of a billing error.

Some paths also show real mathematical or formalization barriers. An AM-GM path repeatedly submitted an introduction while leaving the product bound unfinished. An induction on product divisibility remained partial. Another proof reached a useful logarithmic reduction but struggled with casts and positivity. These failures are more informative than a generic “out of budget” label, yet still concern particular sampled paths.

The feedback diagnostic gave another caution. A Terra arm reached six solves, but the extra goal feedback was never emitted in its apparently improved cells. The result cannot be credited to that unexposed patch. Empty focused goals can genuinely hide remaining obligations; this experiment did not establish that the new visibility interface caused its gains.

The actionable direction is to distinguish **an unaffordable next reservation, an unproductive next step, and a genuinely useful opportunity to complete the proof**. Safer, tighter admission and more reliable submission deserve investigation without simply raising every ceiling.

13. The mistakes worth understanding in detail

Missing terminal evidence can teach the wrong lesson. On amc12_2000_p6, the model’s submitted ending involved omega and norm_num, while assisted verification had actually completed an earlier prefix with nia. Luna’s reflection credited the unexecuted ending. A saved check accepts the nia witness and rejects the proposed norm_num in * ending. Seven of thirty-three successful source histories omit the final verified tactic from the last submitted text; that is an audit flag, not proof of seven identical semantic mistakes. Reflection should receive the exact accepted proof and assistance receipt.

Grounding a name does not validate a snippet. A Terra reducer introduced a have ... := Rle_0_sqr _form where the checked source used valid assert. The lemma exists, so a name-availability gate passes it. The defect enters during reduction. Preserving the checked snippet is more direct than hoping a later model repairs it. Even a checked snippet still needs the right environment and bindings to apply elsewhere.

Parsing can hide already-paid useful work. Concatenated final JSON messages blocked candidate delivery in the structured arms. The corrected adapter recovers one useful saved continuation. That supports fixing the interface and testing its corrected behavior, while keeping the original failed runs in the record.

Observability is not completion. An empty focused-goal list can coexist with unfocused or existential obligations. Only final verifier acceptance establishes a solved theorem. A corrected goal view is useful infrastructure; it needs exposure and outcome evidence before receiving credit for a benchmark gain.

Compaction can destroy the memory that prevents repetition. Some shorter-history paths repeatedly searched for the same facts and never submitted a proof. Measure total requests and progress, not only prompt length. Replay-generated log events must also be separated from actual paid dispatches.

Reporting mistakes can create plausible effects. An old model-family pricing fallback materially understated bills. A prefix-limited YAML reader could mistake embedded evidence for the current outcome. Pooled endpoint comparisons changed direction under pairing. A rendering advantage disappeared across books. These corrections show why small effects need reliable accounting and compatible baselines before interpretation.

The common requirement is a complete evidence chain: **what was proposed; what was executed; what was accepted; what the next role saw; whether the intended treatment ran; and what was finally paid.** Better bookkeeping here is not administrative polish. It determines which research conclusions the experiment can support.

14. What to claim, and what to do next

You can defend an extensive ACE implementation, a systematic series of positive and negative experiments, an accepted budgeting trade-off, and concrete diagnoses of how context, verification, and resource control interact. The recent matched Luna result is promising. A large general ACE coverage benefit, a stronger-model “unlock,” and an intrinsic capability ceiling remain unestablished.

For the next **\$30 of experimental API spending**, I would prioritize the following work. These are starting hypotheses for the next planning session, not additional experiments already performed.

First: admission and proof completion. Establish which failures lose a useful opportunity because of conservative reservations, repeated ineffective operations, or failure to return to submission. Compare the current bounded program with compatible earlier behavior. Use the existing Delphyne budget and search mechanisms. A proposed improvement must preserve financial accounting and identify the newly admitted or redirected work. Increasing all allowances would answer a different question.

Second: terminal evidence and checked advice. Verify the actual accepted proof and assistance receipt reach the Reflector, and that executable advice survives reduction with its checked provenance. Start with offline witnesses. A correctness repair is valuable even before a full benchmark; an end-to-end ACE improvement still requires outcome evidence.

Third: corrected recovery interfaces, if they reach relevant cases. The final-answer parser fix has a useful saved witness. Before paying for a full comparison, show that the corrected mechanism is invoked, returns a candidate, and advances the original obligation. Treat already-corrected code as something to evaluate, not something to “fix” again.

The next session should choose at most two isolated contenders, plan exact problem/seed counts, reuse compatible paid references, and reserve part of the \$30 for failures or justified follow-up. The spending ceiling is not a target. trainX and validationX remain the only evaluation inputs; the latter is development data. No testX access or protected-failure mining belongs in that plan.

For the advisor, the account can be brief:

“We have a working ACE system, but the clearest win is controlled spending rather than a large general coverage gain. Our latest matched Luna comparison is encouraging and still uncertain. Several failed treatments revealed concrete delivery and evidence defects, while continued attempts show both useful Terra capacity and ongoing budget censoring. I want the next experiments to recover useful proof-search opportunity and improve the evidence chain, with cost and coverage measured together.”

Useful advisor decisions are which trade-offs to emphasize in the thesis, how strongly to frame the implementation contribution versus the ACE hypothesis, and whether any future independent evaluation is necessary. The presentation and rehearsal notes are organized around those decisions.